Java Grundlagen

Datentypen, Operatoren, Kontrollstrukturen und Methoden

Autor

Luis Staudt

20. Dezember 2025

Inhaltsverzeichnis

Inhaltsverzeichnis . . . 2
Java Datentypen . . . 4
Primitive Datentypen . . . 4
int - Ganzzahlen . . . 5
double - Fließkommazahlen . . . 7
boolean - Wahrheitswerte . . . 9
char - Einzelnes Zeichen . . . 11
byte - Sehr kleine Ganzzahlen . . . 13
short - Kleine Ganzzahlen . . . 14
long - Sehr große Ganzzahlen . . . 15
float - Einfache Fließkommazahlen . . . 16
String - Zeichenketten . . . 17
Typkonvertierung . . . 20
Übersicht: Primitive Datentypen . . . 22
Arrays . . . 22
array.length - Länge eines Arrays . . . 24
array[index] - Element lesen . . . 26
array[index] = wert - Element schreiben . . . 28
Häufige Fehler . . . 30
Operatoren in Java . . . 33
Arithmetische Operatoren . . . 33
Vergleichsoperatoren . . . 34
Logische Operatoren . . . 36
Zuweisungsoperatoren . . . 38
Inkrement- und Dekrement-Operatoren . . . 39
String-Konkatenation . . . 40
Operatorrangfolge (Priorität) . . . 41
Kontrollstrukturen in Java . . . 43
If-Bedingungen . . . 43
For-Schleifen . . . 49
While-Schleifen . . . 57
Vergleich: For vs. While . . . 60
Häufige Fehler . . . 61

Grundlegende Methoden in Java . . . 63
String-Methoden . . . 63
Ausgabe auf der Konsole . . . 65
println() - Ausgabe mit Zeilenumbruch . . . 65
print() - Ausgabe ohne Zeilenumbruch . . . 67
ConsoleReader - Eingaben von der Konsole . . . 71
readString() - String einlesen . . . 72
readInt() - Ganzzahl einlesen . . . 73
readDouble() - Kommazahl einlesen . . . 74
readByte() - Sehr kleine Ganzzahl einlesen . . . 75
readShort() - Kleine Ganzzahl einlesen . . . 76
readLong() - Sehr große Ganzzahl einlesen . . . 77
readFloat() - Einfache Kommazahl einlesen . . . 78
readIntArray() - Ganzzahl-Array einlesen . . . 79
readDoubleArray() - Kommazahl-Array einlesen . . . 80
readStringArray() - String-Array einlesen . . . 81
Weitere Array-Methoden . . . 83
Bildverarbeitung . . . 84
show() - Bild anzeigen . . . 85
loadFromFileSystem() - Bild von Festplatte laden . . . 86
loadResource() - Bild aus Projekt laden . . . 87
color() - Farbwert erstellen . . . 89
red(), green(), blue() - Farbkanäle extrahieren . . . 91
Häufige Fehler bei Strings . . . 93

Java Datentypen

Primitive Datentypen

Java arbeitet mit verschiedenen Arten von Daten. Die einfachsten davon heißen pprimitive Datentypen“. Diese speichern direkte Werte wie Zahlen oder Ja/Nein-Antworten.

int - Ganzzahlen

Was ist int?

Der Datentyp `int` speichert ganze Zahlen - also Zahlen ohne Komma (wie 5, -10, oder 1000).

Wertebereich:

- Von: -2.147.483.648
- Bis: 2.147.483.647
- Speichergröße: 32 Bit (4 Bytes)

Syntax:

```
int variablenName = wert;
```

Beispiele:

```
// Deklaration und Initialisierung
int alter = 25;
int temperatur = -5;
int punkte = 0;

// Ohne Initialisierung
int zahl;
zahl = 42;

// Berechnungen
int summe = 10 + 20;          // summe = 30
int differenz = 50 - 15;      // differenz = 35
int produkt = 6 * 7;          // produkt = 42
int quotient = 20 / 4;        // quotient = 5
int rest = 17 % 5;            // rest = 2

// Mit Variablen rechnen
int a = 10;
int b = 3;
int ergebnis = a + b;         // ergebnis = 13

// Inkrement und Dekrement
int zaehler = 5;
zaehler++;                    // zaehler = 6
zaehler--;                    // zaehler = 5

```

```
// Zusammengesetzte Operatoren
int wert = 10;
wert += 5;                    // wert = 15 (entspricht: wert = wert + 5)
wert -= 3;                    // wert = 12
wert *= 2;                    // wert = 24
wert /= 4;                    // wert = 6
```

Worauf achten:

- Wenn du eine int-Zahl durch eine int-Zahl teilst, bekommst du wieder eine int-Zahl: `5 / 2 = 2` (nicht 2.5! Die Nachkommastellen werden einfach abgeschnitten)
- Wenn die Zahl zu groß oder zu klein wird, springt sie auf die andere Seite um (das nennt man Überlauf)
- In einem Array ist der Anfangswert immer `0`
- Mit int kannst du keine Kommazahlen speichern

double - Fließkommazahlen

Was ist double?

Der Datentyp `double` speichert Kommazahlen - also Zahlen mit Nachkommastellen (wie 3.14, -5.5, oder 0.001).

Wertebereich:

- Sehr große und sehr kleine Zahlen mit ca. 15 Dezimalstellen Genauigkeit
- Von: $\pm 4.9 \times 10^{-324}$
- Bis: $\pm 1.7 \times 10^{308}$
- Speichergröße: 64 Bit (8 Bytes)

Syntax:

```
double variablenName = wert;
```

Beispiele:

```
// Deklaration und Initialisierung
double pi = 3.14159;
double temperatur = -5.5;
double gewicht = 75.8;
double ergebnis = 0.0;

// Berechnungen
double summe = 10.5 + 20.3;        // summe = 30.8
double produkt = 2.5 * 4.0;        // produkt = 10.0
double quotient = 7.0 / 2.0;       // quotient = 3.5

// Wichtig: Division mit Nachkommastellen
double a = 5.0;
double b = 2.0;
double teil = a / b;               // teil = 2.5

// int zu double konvertieren
int ganzzahl = 5;
double kommazahl = ganzzahl / 2.0; // kommazahl = 2.5
// ODER
double kommazahl2 = (double) ganzzahl / 2; // kommazahl2 = 2.5

// Wissenschaftliche Notation
```

```
24 double klein = 1.5e-4;              // = 0.00015
25 double gross = 3.2e8;               // = 320000000.0
26
27 // Mathematische Funktionen
28 double wurzel = Math.sqrt(16.0);    // wurzel = 4.0
29 double potenz = Math.pow(2, 3);     // potenz = 8.0
30 double absolut = Math.abs(-5.5);    // absolut = 5.5
```

Worauf achten:

- In Java schreibst du einen Punkt statt Komma: `3.14` ist richtig, `3,14` ist falsch
- Computer können nicht immer exakt rechnen: manchmal kommt bei `0.1 + 0.2` nicht genau `0.3` heraus
- Wichtig: `5 / 2` ergibt `2`, aber `5.0 / 2.0` ergibt `2.5` - mindestens eine Zahl muss eine Kommazahl sein
- In einem Array ist der Anfangswert immer `0.0`
- Für Geld solltest du andere Datentypen verwenden (z.B. `BigDecimal`), weil Rundungsfehler problematisch sein können

boolean - Wahrheitswerte

Was ist boolean?

Der Datentyp `boolean` speichert Wahrheitswerte: entweder `true` (wahr) oder `false` (falsch).

Wertebereich:

- Nur zwei mögliche Werte: `true` oder `false`
- Speichergröße: 1 Bit (theoretisch), praktisch abhängig von JVM

Syntax:

```
boolean variablenName = true/false;
```

Beispiele:

```
// Deklaration und Initialisierung
boolean istWahr = true;
boolean istFalsch = false;
boolean hatBestanden = true;

// Vergleichsoperatoren ergeben boolean
int alter = 18;
boolean istErwachsen = (alter >= 18);      // true
boolean istKind = (alter < 12);            // false
boolean istGenau18 = (alter == 18);        // true
boolean istNicht18 = (alter != 18);        // false

// Logische Operatoren
boolean a = true;
boolean b = false;

boolean und = a && b;                      // false (beide muessen true
    sein)
boolean oder = a || b;                     // true (mindestens einer true)
boolean nicht = !a;                        // false (Negation)

// In Bedingungen
int punkte = 85;
boolean bestanden = punkte >= 50;
if (bestanden) {
    System.out.println("Bestanden!");
}
```

```java

// Kombinierte Bedingungen
int temperatur = 25;
boolean angenehm = temperatur >= 18 && temperatur <= 28;

// Boolean in Schleifen
boolean weitermachen = true;
while (weitermachen) {
    // Code
    weitermachen = false;  // Schleife beenden
}

// Vergleich mit Strings
String name = "Max";
boolean istMax = name.equals("Max");       // true
```

Worauf achten:

- Es gibt nur zwei Werte: `true` (wahr) oder `false` (falsch) - keine Zahlen!
- Alles klein schreiben: `true` und `false`, nicht `True` oder `False`
- In einem Array ist der Anfangswert immer `false`
- Zum Vergleichen: Bei Zahlen `==`, bei Text `.equals()`

char - Einzelnes Zeichen

Was ist char?

Der Datentyp `char` speichert genau ein einzelnes Zeichen (wie 'A', '5' oder '!').

Wertebereich:

- Alle Unicode-Zeichen (Buchstaben, Ziffern, Sonderzeichen)
- Von: 0 bis 65.535 (als Zahlenwert)
- Speichergröße: 16 Bit (2 Bytes)

Syntax:

```
char variablenName = 'Zeichen';  // EINFACHE Anfuehrungszeichen!
```

Beispiele:

```
// Deklaration und Initialisierung
char buchstabe = 'A';
char ziffer = '5';
char sonderzeichen = '!';
char leerzeichen = ' ';

// Mit Unicode-Werten
char euro = '\u20AC';                    //    -Symbol
char herzchen = '\u2764';                //

// Escape-Sequenzen
char neuzeile = '\n';                    // Zeilenumbruch
char tab = '\t';                         // Tabulator
char backslash = '\\';                   // \
char anfuehrung = '\'';                  // '

// Zeichen vergleichen
char note = 'A';
if (note == 'A') {
    System.out.println("Sehr gut!");
}

// Zeichen in Berechnungen (ASCII/Unicode-Werte)
char a = 'A';                            // Unicode-Wert: 65
char b = 'B';                            // Unicode-Wert: 66
int differenz = b - a;                   // differenz = 1
```

```java

// Kleinbuchstabe zu Gro buchstabe
char klein = 'a';                          // Unicode-Wert: 97
char gross = (char) (klein - 32);          // gross = 'A' (Wert: 65)

// Pr fen ob Buchstabe
char zeichen = 'M';
boolean istBuchstabe = (zeichen >= 'A' && zeichen <= 'Z') ||
                       (zeichen >= 'a' && zeichen <= 'z');

// Pr fen ob Ziffer
char z = '7';
boolean istZiffer = (z >= '0' && z <= '9');

// char aus String extrahieren
String text = "Hallo";
char erstes = text.charAt(0);              // erstes = 'H'
```

Worauf achten:

- Immer einfache Anführungszeichen verwenden: `'A'` ist richtig, Ä“ ist falsch
- Nur genau ein Zeichen erlaubt: `'A'` funktioniert, `'AB'` nicht
- `char` ist etwas anderes als `String` - ein char ist nur ein einzelnes Zeichen
- In einem Array ist der Anfangswert das NNull-Zeichen“(ein unsichtbares Zeichen)
- Jedes Zeichen hat intern eine Nummer - deshalb kannst du mit char auch rechnen

byte - Sehr kleine Ganzzahlen

Was ist byte?

Der Datentyp `byte` speichert sehr kleine ganze Zahlen. Wird selten benutzt, meist nur wenn man Speicherplatz sparen muss.

Wertebereich:

- Von: -128
- Bis: 127
- Speichergröße: 8 Bit (1 Byte)

Syntax:

```
byte variablenName = wert;
```

Beispiele:

```
// Deklaration und Initialisierung
byte alter = 25;
byte temperatur = -15;
byte prozent = 100;

// H ufig bei Datei-Operationen
byte[] daten = new byte[1024];

// Konvertierung von int zu byte
int gross = 100;
byte klein = (byte) gross;

// Vorsicht bei  berschreitung
byte max = 127;
byte ueberlauf = (byte) (max + 1);         // ueberlauf = -128 (Overflow!)
```

Worauf achten:

- Sehr kleiner Wertebereich: nur -128 bis 127
- Wenn die Zahl zu groß wird, springt sie um (Überlauf)
- Wird meistens nur für spezielle Dinge verwendet
- In einem Array ist der Anfangswert immer `0`

short - Kleine Ganzzahlen

Was ist short?

Der Datentyp `short` speichert kleine ganze Zahlen. Ein Mittelweg zwischen `byte` und `int`.

Wertebereich:

- Von: -32.768
- Bis: 32.767
- Speichergröße: 16 Bit (2 Bytes)

Syntax:

```
short variablenName = wert;
```

Beispiele:

```
// Deklaration und Initialisierung
short jahr = 2024;
short distanz = 5000;

// Konvertierung
int gross = 1000;
short klein = (short) gross;
```

Worauf achten:

- Wird selten verwendet, meist wird direkt `int` genommen
- Wertebereich: -32.768 bis 32.767
- Standard-Wert bei Arrays: `0`

long - Sehr große Ganzzahlen

Was ist long?

Der Datentyp `long` speichert sehr große ganze Zahlen - noch größer als int.

Wertebereich:

- Von: -9.223.372.036.854.775.808
- Bis: 9.223.372.036.854.775.807
- Speichergröße: 64 Bit (8 Bytes)

Syntax:

```
long variablenName = wert;
long grosseZahl = wertL;  // L am Ende fuer long-Literal
```

Beispiele:

```
// Deklaration und Initialisierung
long weltbevoelkerung = 8000000000L;       // L am Ende!
long millisekunden = System.currentTimeMillis();

// Ohne L wird es als int interpretiert (Fehler bei gro en Zahlen)
// long falsch = 8000000000;               // FEHLER!
long richtig = 8000000000L;                // Korrekt

// Berechnungen
long summe = 1000000000L + 2000000000L;
```

Worauf achten:

- Bei großen Zahlen musst du ein `L` ans Ende schreiben: `1000000000L`
- Ohne `L` denkt Java, es ist eine int-Zahl, und das kann schiefgehen
- In einem Array ist der Anfangswert immer `0L`
- Benutze long, wenn deine Zahlen größer als etwa 2 Milliarden werden können

float - Einfache Fließkommazahlen

Was ist float?

Der Datentyp `float` speichert Fließkommazahlen mit einfacher Genauigkeit. Weniger genau als `double`.

Wertebereich:

- Ca. 6-7 Dezimalstellen Genauigkeit
- Von: $\pm 1.4 \times 10^{-45}$
- Bis: $\pm 3.4 \times 10^{38}$
- Speichergröße: 32 Bit (4 Bytes)

Syntax:

```
float variablenName = wertF;  // F am Ende fuer float-Literal
```

Beispiele:

```
// Deklaration und Initialisierung
float pi = 3.14F;                          // F am Ende!
float temperatur = 36.5F;
float gewicht = 75.8F;

// Ohne F wird es als double interpretiert
// float falsch = 3.14;                    // Warnung/Fehler
float richtig = 3.14F;                     // Korrekt

// Berechnungen
float summe = 10.5F + 20.3F;
```

Worauf achten:

- `F` oder `f` am Ende anhängen: `3.14F`
- Weniger genau als `double`
- In der Praxis wird meist `double` bevorzugt
- Standard-Wert bei Arrays: `0.0F`

String - Zeichenketten

Was ist ein String?

Beschreibung

Ein `String` speichert Text - also mehrere Zeichen zusammen (wie "Halloöder MMax Mustermann"). String ist etwas Besonderes: Er gehört nicht zu den primitiven Datentypen.

Eigenschaften:

- Wenn du einen String einmal erstellt hast, kannst du ihn nicht mehr ändern - nur einen neuen erstellen
- Strings schreibst du mit doppelten Anführungszeichen: `TText"`
- Ein String kann so lang sein wie du willst

Syntax:

```
String variablenName = "Text";
```

Beispiele:

```
// Deklaration und Initialisierung
String name = "Max Mustermann";
String begruessung = "Hallo Welt!";
String leer = "";                          // Leerer String
String mitZahl = "Ich bin 25 Jahre alt";

// String-Konkatenation (Verkettung)
String vorname = "Max";
String nachname = "Mustermann";
String vollername = vorname + " " + nachname;  // "Max Mustermann"

// Mit Zahlen verketten
int alter = 25;
String text = "Ich bin " + alter + " Jahre alt";

// String-Methoden
String wort = "Hallo";
int laenge = wort.length();                // 5
char erstes = wort.charAt(0);              // 'H'
String gross = wort.toUpperCase();         // "HALLO"
String klein = wort.toLowerCase();         // "hallo"

```

```java
// Strings vergleichen
String a = "Hallo";
String b = "Hallo";
boolean gleich = a.equals(b);              // true
boolean gleichIgnoreCase = a.equalsIgnoreCase("HALLO");  // true

// WICHTIG: Nicht mit == vergleichen!
String s1 = "Test";
String s2 = new String("Test");
boolean falsch = (s1 == s2);               // false (vergleicht Referenzen)
boolean richtig = s1.equals(s2);           // true (vergleicht Inhalt)

// Teilstrings
String text2 = "Hallo Welt";
String teil = text2.substring(0, 5);       // "Hallo"
String ab = text2.substring(6);            // "Welt"

// Suchen in Strings
String satz = "Java ist toll";
boolean enthaelt = satz.contains("ist");   // true
int position = satz.indexOf("ist");        // 5
boolean beginnt = satz.startsWith("Java"); // true
boolean endet = satz.endsWith("toll");     // true

// Ersetzen
String original = "Hallo Welt";
String neu = original.replace("Welt", "Java");  // "Hallo Java"

// Leerzeichen entfernen
String mitLeer = "  Text  ";
String ohneLeer = mitLeer.trim();          // "Text"

// String in Array umwandeln
String liste = "Apfel,Birne,Orange";
String[] fruechte = liste.split(",");      // ["Apfel", "Birne", "Orange"]

// Zahlen zu String
int zahl = 42;
String zahlString = String.valueOf(zahl);  // "42"
// ODER
String zahlString2 = "" + zahl;            // "42"

// String zu Zahl
String nummerText = "123";
int nummer = Integer.parseInt(nummerText); // 123
double komma = Double.parseDouble("3.14"); // 3.14
```

Worauf achten:

- Doppelte Anführungszeichen verwenden: `TText“` ist richtig, `'Text'` ist falsch
- Zum Vergleichen von Strings immer `.equals()` benutzen, nicht `==`
- String schreibst du immer mit großem S am Anfang
- Wenn du einen String ändern willst, bekommst du eigentlich einen neuen String zurück
- `null` bedeutet “gar kein Text“, ist lleerer Text das ist ein Unterschied
- In einem Array ist der Anfangswert immer `null`

Typkonvertierung

Implizite Konvertierung (Automatisch)

Beschreibung

Java wandelt automatisch kleinere Zahlen in größere um. Zum Beispiel wird eine int-Zahl automatisch zu double, wenn nötig.

Beispiele:

```
// int zu double (automatisch)
int ganzzahl = 5;
double kommazahl = ganzzahl;                // kommazahl = 5.0

// byte zu int (automatisch)
byte klein = 100;
int gross = klein;                          // gross = 100

// char zu int (automatisch)
char buchstabe = 'A';
int code = buchstabe;                       // code = 65 (Unicode-Wert)

// In Berechnungen
int a = 5;
double b = 2.5;
double ergebnis = a + b;                    // ergebnis = 7.5
```

Explizite Konvertierung (Type Cast)

Beschreibung

Wenn du eine größere Zahl in eine kleinere umwandeln willst, musst du Java das ausdrücklich sagen. Dabei können Informationen verloren gehen!

Syntax:

```
zielTyp variable = (zielTyp) quellVariable;
```

Beispiele:

```
// double zu int (Nachkommastellen gehen verloren!)
double kommazahl = 3.7;
int ganzzahl = (int) kommazahl;             // ganzzahl = 3 (nicht 4!)

```

```
// long zu int
long gross = 1000L;
int klein = (int) gross;

// int zu byte (Vorsicht bei gro en Zahlen!)
int zahl = 200;
byte kleinByte = (byte) zahl;              // Overflow!

// Division mit Cast
int a = 5;
int b = 2;
double ergebnis = (double) a / b;          // ergebnis = 2.5

// Runden statt Abschneiden
double wert = 3.7;
int gerundet = (int) Math.round(wert);     // gerundet = 4
```

Worauf achten:

- Wenn du double zu int umwandelst, werden die Nachkommastellen einfach weggeschnitten, nicht gerundet! (3.9 wird zu 3, nicht zu 4)
- Wenn die Zahl zu groß ist, können seltsame Werte herauskommen
- Zum Runden benutze `Math.round()`
- Automatisch: klein zu groß (z.B. int zu double)
- Mit Klammern: groß zu klein (z.B. double zu int)

Übersicht: Primitive Datentypen

Typ	Größe	Wertebereich	Standard
byte	8 Bit	-128 bis 127	0
short	16 Bit	-32.768 bis 32.767	0
int	32 Bit	-2.147.483.648 bis 2.147.483.647	0
long	64 Bit	ca. -9,2 $\times$ 10^{18} bis 9,2 $\times$ 10^{18}	0L
float	32 Bit	ca. $\pm$3,4 $\times$ 10^{38}	0.0F
double	64 Bit	ca. $\pm$1,7 $\times$ 10^{308}	0.0
char	16 Bit	0 bis 65.535 (Unicode)	'\u0000'
boolean	1 Bit	true oder false	false

Tabelle 1: Übersicht der primitiven Datentypen in Java

Arrays

Was ist ein Array?

Beschreibung

Ein Array ist wie eine Liste mit nummerierten Fächern. Jedes Fach kann einen Wert speichern. Wichtig: Alle Werte müssen vom gleichen Typ sein (z.B. nur Zahlen oder nur Text). Die Anzahl der Fächer ist fest.

Eigenschaften:

- Die Größe ist fest: Wenn du ein Array mit 5 Plätzen erstellst, bleiben es 5 Plätze
- Die Nummerierung beginnt bei 0: Das erste Fach hat die Nummer 0, das letzte die Nummer `length - 1`
- Alle Werte im Array müssen vom gleichen Typ sein
- Neue Arrays sind automatisch mit Startwerten gefüllt (0 für Zahlen, false für boolean, null für Text)

Syntax:

```
// Deklaration und Initialisierung mit  G r   e
datentyp[] arrayName = new datentyp[groesse];

// Deklaration und Initialisierung mit Werten
datentyp[] arrayName = {wert1, wert2, wert3};
```

Beispiele:

```
// Array mit fester G r   e erstellen
int[] zahlen = new int[5];        // 5 Pl tze, alle 0
String[] namen = new String[3];   // 3 Pl tze, alle null
boolean[] flags = new boolean[4]; // 4 Pl tze, alle false

// Array mit Werten erstellen
int[] noten = {1, 2, 3, 2, 1};
String[] wochentage = {"Mo", "Di", "Mi", "Do", "Fr"};
double[] preise = {9.99, 14.50, 7.25};

// Mehrdimensionale Arrays
int[][] matrix = new int[3][3];   // 3x3 Matrix
int[][] tabelle = {{1, 2}, {3, 4}, {5, 6}};
```

Worauf achten:

- Die Größe kannst du nach dem Erstellen nicht mehr ändern
- Die Nummerierung beginnt bei 0, nicht bei 1!
- Alle Werte müssen vom gleichen Typ sein
- Wenn du ein Array noch nicht erstellt hast, ist es `null`

array.length - Länge eines Arrays

Was ist array.length?

Mit `length` kannst du herausfinden, wie viele Plätze das Array hat. Wichtig: Bei Arrays schreibst du **keine** Klammern dahinter!

Eigenschaften:

- Ist ein Attribut, keine Methode: **ohne** Klammern **( )**
- Gibt die Größe zurück, mit der das Array erstellt wurde
- Funktioniert auch bei leeren Arrays (nur Standard-Werte)

Syntax:

```
int laenge = arrayName.length;  // OHNE Klammern!
```

Beispiele:

```
// L nge verschiedener Arrays
int[] zahlen = {10, 20, 30, 40, 50};
int anzahl = zahlen.length;                // anzahl = 5

String[] namen = new String[10];
int groesse = namen.length;                // groesse = 10 (auch wenn leer!)

// Leeres Array
int[] leer = new int[0];
int laenge = leer.length;                  // laenge = 0

// In Schleife verwenden
int[] werte = {1, 2, 3, 4};
for (int i = 0; i < werte.length; i++) {
    System.out.println(werte[i]);
}

// Letztes Element zugreifen
int[] liste = {100, 200, 300};
int letzter = liste[liste.length - 1];   // letzter = 300

// Bei mehrdimensionalen Arrays
int[][] matrix = new int[3][4];
int zeilen = matrix.length;                // zeilen = 3
int spalten = matrix[0].length;            // spalten = 4
```

Worauf achten:

- Bei Arrays: `.length` ohne Klammern
- Bei Strings: `.length()` mit Klammern - das ist ein häufiger Fehler!
- Gibt die Anzahl der Plätze zurück, nicht die Anzahl der gefüllten Plätze
- Auch wenn alle Werte noch 0 oder null sind, hat das Array trotzdem seine feste Länge

array[index] - Element lesen

Was ist array[index]?

Mit eckigen Klammern `[]` kannst du auf ein bestimmtes Fach im Array zugreifen. Die Zahl in den Klammern ist die Nummer des Fachs (beginnt bei 0).

Eigenschaften:

- Index beginnt bei 0: Erstes Element ist `array[0]`
- Letztes Element ist `array[array.length - 1]`
- Bei ungültigem Index: `ArrayIndexOutOfBoundsException`

Syntax:

```
datentyp element = arrayName[index];
```

Beispiele:

```
// Einzelne Elemente lesen
int[] zahlen = {10, 20, 30, 40, 50};
int erstes = zahlen[0];                    // erstes = 10
int zweites = zahlen[1];                   // zweites = 20
int letztes = zahlen[4];                   // letztes = 50

// Alle Elemente durchlaufen
String[] woerter = {"Hallo", "Welt", "Java"};
for (int i = 0; i < woerter.length; i++) {
    String wort = woerter[i];
    System.out.println(wort);
}

// Bestimmte Position pr fen
int[] noten = {1, 2, 3, 2, 1};
if (noten[2] == 3) {
    System.out.println("Dritte Note ist eine 3");
}

// Letztes Element
int[] liste = {5, 10, 15, 20};
int letzterWert = liste[liste.length - 1];  // letzterWert = 20

// Element in Berechnung verwenden
int[] preise = {100, 200, 150};
```

```
26 int summe = preise[0] + preise[1] + preise[2];  // summe = 450
27
28 // Mehrdimensionale Arrays
29 int[][] matrix = {{1, 2, 3}, {4, 5, 6}};
30 int wert = matrix[0][1];                  // wert = 2 (1. Zeile, 2. Spalte)
```

Worauf achten:

- Die Nummerierung beginnt bei 0, nicht bei 1!
- Die Nummer muss zwischen 0 und `array.length - 1` liegen
- Wenn die Nummer zu groß oder zu klein ist, stürzt das Programm ab
- Das letzte Fach erreichst du mit `array[array.length - 1]`

array[index] = wert - Element schreiben

Was ist array[index] = wert?

So kannst du einen Wert in ein bestimmtes Fach des Arrays legen (oder den alten Wert überschreiben).

Eigenschaften:

- Überschreibt den vorherigen Wert an dieser Position
- Der neue Wert muss zum Datentyp des Arrays passen
- Index muss gültig sein (0 bis `length - 1`)

Syntax:

```
arrayName[index] = neuerWert;
```

Beispiele:

```
// Array bef llen
int[] zahlen = new int[5];                // Array mit 5 Pl tzen, alle 0
zahlen[0] = 10;                           // Erstes Element auf 10 setzen
zahlen[1] = 20;                           // Zweites Element auf 20 setzen
zahlen[4] = 50;                           // Letztes Element auf 50 setzen
// Ergebnis: {10, 20, 0, 0, 50}

// Werte  ndern
String[] namen = {"Anna", "Bob", "Clara"};
namen[1] = "Max";                         // "Bob" wird zu "Max"
// Ergebnis: {"Anna", "Max", "Clara"}

// In Schleife bef llen
int[] quadrate = new int[5];
for (int i = 0; i < quadrate.length; i++) {
    quadrate[i] = i * i;
}
// Ergebnis: {0, 1, 4, 9, 16}

// Wert erh hen
int[] punkte = {10, 20, 30};
punkte[0] = punkte[0] + 5;                // Erstes Element um 5 erh hen
// ODER k rzer:
punkte[0] += 5;
// Ergebnis: {15, 20, 30}
```

```java

// Array "leeren" (auf Standard-Wert setzen)
int[] werte = {1, 2, 3, 4, 5};
for (int i = 0; i < werte.length; i++) {
    werte[i] = 0;
}
// Ergebnis: {0, 0, 0, 0, 0}

// Zwei Elemente tauschen
int[] liste = {10, 20, 30};
int temp = liste[0];                        // Zwischenspeichern
liste[0] = liste[2];                        // 30 nach vorne
liste[2] = temp;                            // 10 nach hinten
// Ergebnis: {30, 20, 10}

// Array aus Benutzereingabe f llen
int[] zahlenListe = new int[3];
for (int i = 0; i < zahlenListe.length; i++) {
    // zahlenListe[i] = scanner.nextInt();
}

// Mehrdimensionale Arrays
int[][] matrix = new int[2][2];
matrix[0][0] = 1;
matrix[0][1] = 2;
matrix[1][0] = 3;
matrix[1][1] = 4;
// Ergebnis: {{1, 2}, {3, 4}}
```

Worauf achten:

- Die Nummer muss zwischen 0 und `array.length - 1` liegen
- Wenn die Nummer ungültig ist, stürzt das Programm ab
- Der Wert muss zum Typ des Arrays passen (Zahl für int-Array, Text für String-Array usw.)
- Der alte Wert an dieser Stelle geht verloren - er wird komplett ersetzt

Häufige Fehler

String mit == vergleichen

```
// FALSCH
String a = "Hallo";
String b = "Hallo";
if (a == b) { }  // Vergleicht Referenzen, nicht Inhalt!

// RICHTIG
if (a.equals(b)) { }  // Vergleicht Inhalt
```

Integer-Division

```
// FALSCH - ergibt 2, nicht 2.5
int a = 5;
int b = 2;
double ergebnis = a / b;  // ergebnis = 2.0

// RICHTIG
double ergebnis = (double) a / b;  // ergebnis = 2.5
// ODER
double ergebnis = 5.0 / 2.0;  // ergebnis = 2.5
```

char vs. String verwechseln

```
// FALSCH
char buchstabe = "A";  // Fehler: doppelte Anfuehrungszeichen
String wort = 'Hallo';  // Fehler: einfache Anfuehrungszeichen

// RICHTIG
char buchstabe = 'A';   // einfache Anfuehrungszeichen
String wort = "Hallo";  // doppelte Anfuehrungszeichen
```

L und F vergessen

```
// FALSCH
long gross = 5000000000;  // Fehler: zu gross fuer int
float zahl = 3.14;        // Warnung: double wird zu float

// RICHTIG
long gross = 5000000000L;  // L am Ende
float zahl = 3.14F;        // F am Ende
```

Overflow nicht beachten

```
// Problem
byte b = 127;
b++;  // b = -128 (Overflow!)

int i = 2147483647;
i++;  // i = -2147483648 (Overflow!)
```

null vs. leerer String

```
String a = null;      // Keine Referenz (keine Methoden aufrufbar!)
String b = "";        // Leerer String (Methoden aufrufbar)

// a.length();        // FEHLER: NullPointerException
int laenge = b.length();  // Funktioniert: laenge = 0
```

String vs. Array - length

```
// FALSCH
String text = "Hallo";
int laenge = text.length;  // Fehler: Strings brauchen ()

int[] zahlen = {1, 2, 3};
int anzahl = zahlen.length();  // Fehler: Arrays haben KEINE ()

// RICHTIG
int laenge = text.length();    // String MIT Klammern
int anzahl = zahlen.length;    // Array OHNE Klammern
```

Array-Index-Grenzen

```
// FALSCH
int[] zahlen = {10, 20, 30};  // length = 3, Indizes: 0, 1, 2
int x = zahlen[3];            // Fehler! Index 3 existiert nicht
int y = zahlen[-1];           // Fehler! Negative Indizes nicht erlaubt

// RICHTIG
int letzter = zahlen[zahlen.length - 1];  // Index 2
int erster = zahlen[0];                   // Index 0
```

Array-Größe nach Erstellung ändern

```
// FALSCH
int[] zahlen = new int[5];
zahlen.length = 10;  // Fehler! length ist read-only

// RICHTIG - neues Array erstellen und Werte kopieren
int[] zahlen = new int[5];
int[] neuesArray = new int[10];
for (int i = 0; i < zahlen.length; i++) {
    neuesArray[i] = zahlen[i];
}
zahlen = neuesArray;  // Referenz auf neues Array
```

Operatoren in Java

Arithmetische Operatoren

Beschreibung:

Mit diesen Operatoren kannst du rechnen (plus, minus, mal, geteilt).

Operator	Bedeutung	Beispiel	Ergebnis
+	Addition	5 + 3	8
-	Subtraktion	5 - 3	2
*	Multiplikation	5 * 3	15
/	Division	10 / 3	3 (bei int)
%	Modulo (Rest)	10 % 3	1

Beispiele:

```java
int a = 10;
int b = 3;
int summe = a + b;         // summe = 13
int differenz = a - b;     // differenz = 7
int produkt = a * b;       // produkt = 30
int quotient = a / b;      // quotient = 3 (Ganzzahldivision!)
int rest = a % b;          // rest = 1

// Gerade oder ungerade pruefen
int zahl = 7;
if (zahl % 2 == 0) {
    System.out.println("Gerade");
} else {
    System.out.println("Ungerade");
}
```

Wichtig bei Division:

```java
int x = 7 / 2;       // x = 3 (int-Division schneidet ab)
double y = 7.0 / 2; // y = 3.5 (mit Kommazahl)
```

Vergleichsoperatoren

Beschreibung:

Mit diesen Operatoren kannst du Werte vergleichen. Das Ergebnis ist immer `true` (wahr) oder `false` (falsch).

Operator	Bedeutung	Beispiel	Ergebnis
==	Gleich	`5 == 5`	`true`
!=	Ungleich	`5 != 3`	`true`
>	Größer	`5 > 3`	`true`
<	Kleiner	`5 < 3`	`false`
>=	Größer oder gleich	`5 >= 5`	`true`
<=	Kleiner oder gleich	`5 <= 3`	`false`

Beispiele:

```
int a = 10;
int b = 5;

if (a > b) {
    System.out.println("a ist groesser");
}

if (a == 10) {
    System.out.println("a ist 10");
}

if (b != 0) {
    int ergebnis = a / b;  // Division nur wenn b nicht 0 ist
}

// Array-Index pruefen
int[] zahlen = {1, 2, 3};
int index = 2;
if (index >= 0 && index < zahlen.length) {
    System.out.println(zahlen[index]);
}
```

Wichtig bei Strings:

```
String text1 = "Hallo";
String text2 = "Hallo";

// FALSCH - vergleicht Objektreferenzen, nicht Inhalt!
```

```
5 if (text1 == text2) { }
6
7 // RICHTIG - vergleicht Inhalt
8 if (text1.equals(text2)) { }
```

Logische Operatoren

Beschreibung:

Mit diesen Operatoren kannst du mehrere Bedingungen kombinieren (z.B. "wenn A UND B wahr sind").

Operator	Bedeutung	Beispiel	Ergebnis
&&	Und (AND)	`true && false`	`false`
||	Oder (OR)	`true || false`	`true`
!	Nicht (NOT)	`!true`	`false`

Beispiele:

```java
int alter = 20;
boolean hatFuehrerschein = true;

// UND: Beide Bedingungen muessen wahr sein
if (alter >= 18 && hatFuehrerschein) {
    System.out.println("Darf Auto fahren");
}

// ODER: Mindestens eine Bedingung muss wahr sein
int note = 1;
if (note == 1 || note == 2) {
    System.out.println("Gute Note!");
}

// NICHT: Kehrt Wahrheitswert um
boolean gefunden = false;
if (!gefunden) {
    System.out.println("Noch nicht gefunden");
}

// Array-Grenzen pruefen
int[] array = {1, 2, 3};
int i = 1;
if (i >= 0 && i < array.length) {
    System.out.println(array[i]);
}

// Zeichen ist Buchstabe oder Ziffer
char c = 'A';
if ((c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z')) {
    System.out.println("Ist ein Buchstabe");
}
```

Wichtig - Kurzschluss-Auswertung:

```
// Bei && prueft Java die rechte Seite nicht mehr, wenn die linke schon
    false ist
int[] zahlen = {1, 2, 3};
int index = 5;
if (index < zahlen.length && zahlen[index] > 0) {
    // Kein Fehler! zahlen[index] wird gar nicht mehr ausgefuehrt,
    // weil index < zahlen.length bereits false ist
}
```

Zuweisungsoperatoren

Beschreibung:

Mit diesen Operatoren kannst du Werte in Variablen speichern oder ändern.

Operator	Bedeutung	Beispiel	Entspricht
=	Zuweisung	a = 5	-
+=	Addition und Zuweisung	a += 3	a = a + 3
-=	Subtraktion und Zuweisung	a -= 3	a = a - 3
*=	Multiplikation und Zuweisung	a *= 3	a = a * 3
/=	Division und Zuweisung	a /= 3	a = a / 3
%=	Modulo und Zuweisung	a %= 3	a = a % 3

Beispiele:

```
int x = 10;
x += 5;   // x = 15 (entspricht: x = x + 5)
x -= 3;   // x = 12 (entspricht: x = x - 3)
x *= 2;   // x = 24 (entspricht: x = x * 2)
x /= 4;   // x = 6  (entspricht: x = x / 4)
x %= 4;   // x = 2  (entspricht: x = x % 4)

// Summe aufaddieren
int summe = 0;
int[] zahlen = {5, 10, 15};
for (int i = 0; i < zahlen.length; i++) {
    summe += zahlen[i];  // Kuerzer als: summe = summe + zahlen[i]
}

// Zeichen zaehlen
String text = "Hallo";
int anzahl = 0;
for (int i = 0; i < text.length(); i++) {
    if (text.charAt(i) == 'l') {
        anzahl++;  // Entspricht: anzahl = anzahl + 1
    }
}
```

Inkrement- und Dekrement-Operatoren

Beschreibung:

Mit diesen Operatoren kannst du eine Zahl um 1 erhöhen oder verringern.

Operator	Bedeutung	Beispiel	Ergebnis
++	Inkrement (erhöhen um 1)	`i++`	`i = i + 1`
-	Dekrement (verringern um 1)	`i-`	`i = i - 1`

Beispiele:

```
int i = 5;
i++;        // i = 6 (entspricht: i = i + 1)
i--;        // i = 5 (entspricht: i = i - 1)

// In Schleifen
for (int j = 0; j < 5; j++) {  // j wird nach jedem Durchlauf um 1 erhoeht
    System.out.println(j);
}

// Countdown
int countdown = 10;
while (countdown > 0) {
    System.out.println(countdown);
    countdown--;
}

// Zaehler erhoehen
int anzahl = 0;
String text = "Hallo";
for (int k = 0; k < text.length(); k++) {
    if (text.charAt(k) == 'l') {
        anzahl++;
    }
}
```

Unterschied: Vorher oder nachher erhöhen:

```
int a = 5;
int b = a++;  // b = 5, dann a = 6 (++ nachher: erst den Wert nehmen, dann
    erhoehen)

int c = 5;
int d = ++c;  // c = 6, dann d = 6 (++ vorher: erst erhoehen, dann den Wert
     nehmen)
```

String-Konkatenation

Beschreibung:

Der + Operator kann auch Texte zusammenfügen (z.B. “Hallo“+ + “Weltërgibt “Hallo Welt“).

Beispiele:

```
String vorname = "Anna";
String nachname = "Schmidt";
String vollname = vorname + " " + nachname;  // "Anna Schmidt"

// Mit Zahlen
int alter = 25;
String text = "Ich bin " + alter + " Jahre alt";  // "Ich bin 25 Jahre alt"

// In Schleifen
String wort = "Hallo";
String umgekehrt = "";
for (int i = wort.length() - 1; i >= 0; i--) {
    umgekehrt = umgekehrt + wort.charAt(i);
}
// umgekehrt = "ollaH"

// Array-Elemente verbinden
int[] zahlen = {1, 2, 3, 4, 5};
String ausgabe = "";
for (int i = 0; i < zahlen.length; i++) {
    ausgabe += zahlen[i];
    if (i < zahlen.length - 1) {
        ausgabe += ", ";
    }
}
// ausgabe = "1, 2, 3, 4, 5"
```

Operatorrangfolge (Priorität)

Beschreibung:

Wenn du mehrere Operatoren in einer Zeile hast, werden sie in einer bestimmten Reihenfolge ausgeführt (wie in Mathe: PPunkt vor Strich“).

Von höchster zu niedrigster Priorität:

1. **Klammern**: ( )
2. **Inkrement/Dekrement**: ++, -
3. **Negation**: !
4. **Multiplikation, Division, Modulo**: *, /, %
5. **Addition, Subtraktion**: +, -
6. **Vergleichsoperatoren**: <, >, <=, >=
7. **Gleichheit**: ==, !=
8. **Logisches UND**: &&
9. **Logisches ODER**: ||
10. **Zuweisung**: =, +=, -=, etc.

Beispiele:

```
int ergebnis = 5 + 3 * 2;      // ergebnis = 11 (nicht 16!)
// * wird vor + ausgefuehrt: 5 + (3 * 2) = 5 + 6 = 11

int x = (5 + 3) * 2;            // x = 16
// Klammern aendern die Reihenfolge: (5 + 3) * 2 = 8 * 2 = 16

boolean b = 5 > 3 && 2 < 4;     // b = true
// Erst Vergleiche, dann &&: (5 > 3) && (2 < 4) = true && true = true

int y = 10 + 5 * 2 - 3;         // y = 17
// Reihenfolge: 5 * 2 = 10, dann 10 + 10 = 20, dann 20 - 3 = 17

// Bei Unsicherheit: Klammern verwenden!
int z = ((10 + 5) * 2) - 3;     // z = 27 (macht Absicht deutlich)
```

Worauf achten:

- Wenn du dir unsicher bist, setze Klammern! Das macht den Code klarer.
- PPunkt vor Strich“: Mal und Geteilt kommen vor Plus und Minus
- Vergleiche (z.B. >) werden vor logischen Operatoren (z.B. &&) ausgeführt
- Klammern haben immer Vorrang - nutze sie, um die Reihenfolge festzulegen

Kontrollstrukturen in Java

If-Bedingungen

Warum braucht man If-Bedingungen?

Mit If-Bedingungen kann dein Programm Entscheidungen treffen. Zum Beispiel: “Wenn das Alter größer als 18 ist, dann zeige 'volljährig' an.

Wann brauchst du If?

- Wenn du prüfen willst, ob eine Zahl größer, kleiner oder gleich einem Wert ist
- Wenn du zwei Werte vergleichen willst
- Wenn du auf Benutzereingaben reagieren willst
- Wenn du Fehler vermeiden willst (z.B. prüfen, ob ein Array-Index gültig ist)

Einfache If-Bedingung

Syntax:

```
if (Bedingung) {
    Anweisung;
}
```

Beispiele:

```
// Zahl pruefen
int alter = 18;
if (alter >= 18) {
    System.out.println("Volljaehrig");
}

// Array-Index pruefen
int[] zahlen = {10, 20, 30};
int index = 1;
if (index >= 0 && index < zahlen.length) {
    System.out.println(zahlen[index]);  // Sicher zugreifen
}

// Zeichen pruefen
String text = "Hallo";
if (text.charAt(0) == 'H') {
    System.out.println("Beginnt mit H");
}

// String-Laenge pruefen
String wort = "Test";
if (wort.length() > 3) {
    System.out.println("Wort ist laenger als 3 Zeichen");
}

// Wert gefunden
boolean gefunden = false;
int[] liste = {5, 10, 15, 20};
int gesucht = 15;
for (int i = 0; i < liste.length; i++) {
    if (liste[i] == gesucht) {
        gefunden = true;
    }
}
```

If-Else-Bedingung

Syntax:

```
if (Bedingung) {
    Anweisung1;
} else {
    Anweisung2;
}
```

Beispiele:

```
// Gerade oder ungerade
int zahl = 7;
if (zahl % 2 == 0) {
    System.out.println("Gerade");
} else {
    System.out.println("Ungerade");
}

// Positiv oder negativ
int wert = -5;
if (wert >= 0) {
    System.out.println("Positiv oder Null");
} else {
    System.out.println("Negativ");
}

// Array leer oder gefuellt
int[] zahlen = {1, 2, 3};
if (zahlen.length > 0) {
    System.out.println("Array hat " + zahlen.length + " Elemente");
} else {
    System.out.println("Array ist leer");
}

// Zeichen vorhanden
String text = "Java";
if (text.length() > 0) {
    char erstes = text.charAt(0);
    System.out.println("Erstes Zeichen: " + erstes);
} else {
    System.out.println("String ist leer");
}
```

If-Else-If-Bedingung

Syntax:

```
if (Bedingung1) {
    Anweisung1;
} else if (Bedingung2) {
    Anweisung2;
} else {
    Anweisung3;
}
```

Beispiele:

```
// Notenbewertung
int punkte = 85;
if (punkte >= 90) {
    System.out.println("Note: Sehr gut");
} else if (punkte >= 80) {
    System.out.println("Note: Gut");
} else if (punkte >= 70) {
    System.out.println("Note: Befriedigend");
} else {
    System.out.println("Note: Nicht bestanden");
}

// Groessenvergleich
int a = 5;
int b = 10;
if (a > b) {
    System.out.println("a ist groesser");
} else if (a < b) {
    System.out.println("b ist groesser");
} else {
    System.out.println("a und b sind gleich");
}

// String-Laenge kategorisieren
String eingabe = "Hallo";
if (eingabe.length() == 0) {
    System.out.println("Leer");
} else if (eingabe.length() < 5) {
    System.out.println("Kurz");
} else if (eingabe.length() < 10) {
    System.out.println("Mittel");
} else {
```

```
33     System.out.println("Lang");
34 }
35
36 // Index-Position bestimmen
37 int[] werte = {10, 20, 30, 40, 50};
38 int index = 2;
39 if (index < 0) {
40     System.out.println("Index zu klein");
41 } else if (index >= werte.length) {
42     System.out.println("Index zu gross");
43 } else {
44     System.out.println("Wert: " + werte[index]);
45 }
```

Verschachtelte If-Bedingungen

Syntax:

```
if (Bedingung1) {
    if (Bedingung2) {
        Anweisung;
    }
}
```

Beispiele:

```
// Doppelte Pruefung
String text = "Test";
if (text.length() > 0) {
    if (text.charAt(0) == 'T') {
        System.out.println("Beginnt mit T und ist nicht leer");
    }
}

// Array-Zugriff mit Pruefung
int[] zahlen = {5, 10, 15, 20};
int index = 2;
if (index >= 0 && index < zahlen.length) {
    if (zahlen[index] > 10) {
        System.out.println("Wert an Position " + index + " ist groesser als
    10");
    }
}

// Mehrere Bedingungen
int zahl = 24;
if (zahl > 0) {
    if (zahl % 2 == 0) {
        if (zahl < 100) {
            System.out.println("Positive gerade Zahl unter 100");
        }
    }
}
```

For-Schleifen

Warum braucht man For-Schleifen?

Mit For-Schleifen kannst du Code mehrmals ausführen lassen. Das ist nützlich, wenn du:

- Durch alle Werte in einem Array gehen willst
- Jeden Buchstaben in einem Text prüfen willst
- Etwas eine bestimmte Anzahl wiederholen willst (z.B. 10-mal)
- Array-Werte ändern willst (dafür brauchst du die Position/Index)

Standard For-Schleife

Syntax:

```
for (Initialisierung; Bedingung; Inkrement) {
    Anweisung;
}
```

Wichtig:

Du kannst alle drei Teile (Start, Bedingung, Erhöhung) auch weglassen - das ist erlaubt, aber für Anfänger meist nicht nötig.

Typisches Muster für Arrays:

```
for (int i = 0; i < array.length; i++) {
    // Verwende array[i]
}
```

Typisches Muster für Strings:

```
for (int i = 0; i < text.length(); i++) {
    // Verwende text.charAt(i)
}
```

Beispiele:

```
// Zahlen von 0 bis 4 ausgeben
for (int i = 0; i < 5; i++) {
    System.out.println(i);  // Gibt aus: 0, 1, 2, 3, 4
}

// Array durchlaufen
int[] zahlen = {10, 20, 30, 40, 50};
for (int i = 0; i < zahlen.length; i++) {
    System.out.println(zahlen[i]);
}

// String durchlaufen
String text = "Hallo";
for (int i = 0; i < text.length(); i++) {
    char c = text.charAt(i);
    System.out.println(c);
}
```

```java

// Array-Werte aendern
int[] werte = {1, 2, 3, 4, 5};
for (int i = 0; i < werte.length; i++) {
    werte[i] = werte[i] * 2;  // Jeden Wert verdoppeln
}
// Ergebnis: {2, 4, 6, 8, 10}

// Array befuellen
int[] quadrate = new int[5];
for (int i = 0; i < quadrate.length; i++) {
    quadrate[i] = i * i;
}
// Ergebnis: {0, 1, 4, 9, 16}

// Summe berechnen
int[] liste = {5, 10, 15, 20};
int summe = 0;
for (int i = 0; i < liste.length; i++) {
    summe = summe + liste[i];
}
System.out.println("Summe: " + summe);  // Summe: 50

// Zeichen zaehlen
String wort = "Banane";
int anzahlA = 0;
for (int i = 0; i < wort.length(); i++) {
    if (wort.charAt(i) == 'a') {
        anzahlA++;
    }
}
System.out.println("Anzahl 'a': " + anzahlA);  // Anzahl 'a': 3

// Rueckwaerts durchlaufen
int[] zahlen2 = {1, 2, 3, 4, 5};
for (int i = zahlen2.length - 1; i >= 0; i--) {
    System.out.println(zahlen2[i]);  // Gibt aus: 5, 4, 3, 2, 1
}

// Nur gerade Indizes
int[] daten = {10, 20, 30, 40, 50, 60};
for (int i = 0; i < daten.length; i = i + 2) {
    System.out.println(daten[i]);  // Gibt aus: 10, 30, 50
}

// Teilstring durchsuchen (von Position 2 bis Ende)
String satz = "Hallo Welt";
```

```
for (int i = 2; i < satz.length(); i++) {
    System.out.println(satz.charAt(i));  // Gibt aus: l, l, o, , W, e, l, t
}
```

Optionale Teile der For-Schleife

Alle drei Teile können weggelassen werden!

Initialisierung weglassen

Wenn die Variable bereits vorher deklariert wurde:

```
// Beispiel 1: Zaehler bereits vorhanden
int i = 0;
for (; i < 5; i++) {
    System.out.println(i);
}

// Beispiel 2: Start-Index von aussen vorgegeben
int startIndex = 2;
for (; startIndex < 10; startIndex++) {
    System.out.println(startIndex);
}
```

Bedingung weglassen

Wenn die Bedingung immer wahr sein soll (Endlosschleife oder mit `break` beenden):

```
// Beispiel 1: Endlosschleife mit break
int[] zahlen = {10, 20, 30, 40, 50};
for (int i = 0; ; i++) {
    if (i >= zahlen.length) {
        break;  // Schleife manuell beenden
    }
    System.out.println(zahlen[i]);
}

// Beispiel 2: Suche mit vorzeitigem Abbruch
String text = "Hallo Welt";
for (int i = 0; ; i++) {
    if (i >= text.length()) {
        break;
    }
    if (text.charAt(i) == 'W') {
        System.out.println("Gefunden an Position: " + i);
        break;
    }
}
```

Inkrement weglassen

Wenn die Erhöhung manuell im Schleifenkörper erfolgt:

```java
// Beispiel 1: Variable Schrittweite
int[] zahlen = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
for (int i = 0; i < zahlen.length; ) {
    System.out.println(zahlen[i]);
    if (zahlen[i] % 2 == 0) {
        i = i + 1;  // Nach geraden Zahlen: 1 Schritt
    } else {
        i = i + 2;  // Nach ungeraden Zahlen: 2 Schritte
    }
}

// Beispiel 2: Bedingte Erhoehung
String text = "aabbbcccc";
for (int i = 0; i < text.length(); ) {
    char c = text.charAt(i);
    System.out.println("Zeichen: " + c);
    // Ueberspringe alle gleichen Zeichen
    while (i < text.length() && text.charAt(i) == c) {
        i++;  // Erhoehung hier statt im for-Kopf
    }
}
```

Mehrere Teile weglassen

```java
// Alle Teile weggelassen = Endlosschleife
// for (;;) {
//     System.out.println("Laeuft unendlich");
// }

// Nur Bedingung vorhanden
int i = 0;
for (; i < 5; ) {
    System.out.println(i);
    i++;
}
```

Verschachtelte For-Schleifen

Syntax:

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        Anweisung;
    }
}
```

Beispiele:

```
// Multiplikationstabelle
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        System.out.println(i + " * " + j + " = " + (i * j));
    }
}

// Zwei Arrays vergleichen
int[] array1 = {1, 2, 3};
int[] array2 = {2, 3, 4};
for (int i = 0; i < array1.length; i++) {
    for (int j = 0; j < array2.length; j++) {
        if (array1[i] == array2[j]) {
            System.out.println("Gemeinsamer Wert: " + array1[i]);
        }
    }
}

// String-Vergleich (naiv)
String text1 = "abc";
String text2 = "bcd";
for (int i = 0; i < text1.length(); i++) {
    for (int j = 0; j < text2.length(); j++) {
        if (text1.charAt(i) == text2.charAt(j)) {
            System.out.println("Gemeinsames Zeichen: " + text1.charAt(i));
        }
    }
}

// Pattern ausgeben
for (int i = 1; i <= 4; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.print("*");
    }
```

```
35     System.out.println();
36 }
37 // Ausgabe:
38 // *
39 // **
40 // ***
41 // ****
```

While-Schleifen

Warum braucht man While-Schleifen?

While-Schleifen benutzt du, wenn:

- Du nicht weißt, wie oft du wiederholen musst
- Die Schleife laufen soll, bis etwas Bestimmtes passiert
- Du ßo lange wie nötig“wiederholen willst
- Du nach etwas suchst und aufhören willst, sobald du es gefunden hast

Unterschied zur For-Schleife:

- For-Schleife: “Wiederhole genau 10-malöder “Gehe durch alle Array-Werte“
- While-Schleife: “Wiederhole, solange die Bedingung wahr ist“

While-Schleife

Syntax:

```
while (Bedingung) {
    Anweisung;
}
```

Beispiele:

```
// Zahlen von 0 bis 4 ausgeben
int i = 0;
while (i < 5) {
    System.out.println(i);
    i++;
}

// Element in Array suchen
int[] zahlen = {10, 20, 30, 40, 50};
int gesucht = 30;
int index = 0;
boolean gefunden = false;
while (index < zahlen.length && !gefunden) {
    if (zahlen[index] == gesucht) {
        gefunden = true;
        System.out.println("Gefunden an Position: " + index);
    }
    index++;
}

// Erste Position eines Zeichens finden
String text = "Hallo Welt";
char gesuchtesZeichen = 'W';
int position = 0;
while (position < text.length()) {
    if (text.charAt(position) == gesuchtesZeichen) {
        System.out.println("Gefunden an Position: " + position);
        break;  // Schleife beenden
    }
    position++;
}

// Solange Wert unter Grenze
int summe = 0;
int zahl = 1;
while (summe < 100) {
```

```
37      summe = summe + zahl;
38      zahl++;
39  }
40  System.out.println("Summe: " + summe);
41  System.out.println("Zahlen addiert: " + (zahl - 1));
42
43  // Array durchlaufen (alternative zu for)
44  int[] werte = {5, 10, 15, 20};
45  int idx = 0;
46  while (idx < werte.length) {
47      System.out.println(werte[idx]);
48      idx++;
49  }
50
51  // Bedingung wird nie erfuellt -> Schleife laeuft nicht
52  int x = 10;
53  while (x < 5) {
54      System.out.println("Wird nicht ausgegeben");
55      x++;
56  }
57
58  // Countdown
59  int countdown = 5;
60  while (countdown > 0) {
61      System.out.println(countdown);
62      countdown--;
63  }
64  System.out.println("Start!");
65
66  // Erstes Vorkommen eines Teilstrings finden
67  String satz = "Der Hund spielt im Garten";
68  String suche = "spielt";
69  int pos = 0;
70  boolean gefundenString = false;
71  while (pos <= satz.length() - suche.length() && !gefundenString) {
72      boolean match = true;
73      for (int j = 0; j < suche.length(); j++) {
74          if (satz.charAt(pos + j) != suche.charAt(j)) {
75              match = false;
76          }
77      }
78      if (match) {
79          gefundenString = true;
80          System.out.println("Gefunden an Position: " + pos);
81      }
82      pos++;
83  }
```

Vergleich: For vs. While

Wann For-Schleife?

```
// Wenn die Anzahl der Durchlaeufe bekannt ist
for (int i = 0; i < 10; i++) { }

// Beim Durchlaufen von Arrays
int[] zahlen = {1, 2, 3, 4, 5};
for (int i = 0; i < zahlen.length; i++) { }

// Beim Durchlaufen von Strings
String text = "Hallo";
for (int i = 0; i < text.length(); i++) { }

// Wenn man den Index braucht
for (int i = 0; i < array.length; i++) {
    array[i] = i * 2;  // Index wird zum Aendern gebraucht
}
```

Wann While-Schleife?

```
// Wenn die Anzahl der Durchlaeufe unbekannt ist
while (summe < 1000) {
    summe = summe + wert;
}

// Bei Suche mit vorzeitigem Abbruch
while (index < array.length && !gefunden) {
    if (array[index] == gesucht) {
        gefunden = true;
    }
    index++;
}

// Bedingungsabhaengige Wiederholung
while (bedingungErfuellt()) {
    macheEtwas();
}
```

Häufige Fehler

Endlosschleifen

```
// FALSCH - Endlosschleife!
int i = 0;
while (i < 5) {
    System.out.println(i);
    // i++ vergessen! Schleife laeuft unendlich
}

// RICHTIG
int i = 0;
while (i < 5) {
    System.out.println(i);
    i++;  // Zaehler erhoehen
}
```

Falsche Bedingungen

```
// FALSCH - ArrayIndexOutOfBoundsException!
int[] zahlen = {1, 2, 3};
for (int i = 0; i <= zahlen.length; i++) {  // <= ist falsch!
    System.out.println(zahlen[i]);
}

// RICHTIG
for (int i = 0; i < zahlen.length; i++) {  // < ist richtig
    System.out.println(zahlen[i]);
}
```

Semikolon-Fehler

```
// FALSCH - Semikolon beendet Schleife sofort
for (int i = 0; i < 5; i++);  // Semikolon hier ist falsch!
{
    System.out.println(i);  // Wird nur einmal ausgefuehrt, i nicht bekannt
}

// RICHTIG
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}
```

Off-by-One Fehler

```
String text = "Hallo";  // length = 5, Indizes: 0-4

// FALSCH
for (int i = 0; i <= text.length(); i++) {  // i = 5 ist zu viel!
    System.out.println(text.charAt(i));
}

// RICHTIG
for (int i = 0; i < text.length(); i++) {
    System.out.println(text.charAt(i));
}

// RICHTIG - Letztes Element direkt
char letztes = text.charAt(text.length() - 1);
```

Grundlegende Methoden in Java

String-Methoden

length() - Länge eines Strings

Was macht die Methode?

Gibt die Anzahl der Zeichen im String zurück.

Syntax:

```
int laenge = text.length();
```

Beispiele:

```
String name = "Anna";
int laenge = name.length();  // laenge = 4

String wort = "Hallo Welt";
int laenge2 = wort.length();  // laenge2 = 10 (Leerzeichen zaehlt mit!)

String leer = "";
int laenge3 = leer.length();  // laenge3 = 0

String satz = "Java";
for (int i = 0; i < satz.length(); i++) {
    // Schleife laeuft 4 mal (von 0 bis 3)
}
```

Worauf achten:

- `length()` zählt alle Zeichen: Der Text "Hi"hat die Länge 2
- Leerzeichen und Sonderzeichen zählen mit
- Bei Strings: `.length()` mit Klammern
- Bei Arrays: `.length` ohne Klammern - das ist ein wichtiger Unterschied!

charAt(int index) - Zeichen an Position

Was macht die Methode?

Gibt dir das Zeichen an einer bestimmten Stelle im Text zurück. Wichtig: Die Zählung beginnt bei 0!

Syntax:

```
char zeichen = text.charAt(index);
```

Beispiele:

```
String wort = "Hallo";
char erstes = wort.charAt(0);   // erstes = 'H'
char zweites = wort.charAt(1);  // zweites = 'a'
char letztes = wort.charAt(4);  // letztes = 'o'

// Alle Zeichen durchlaufen
String text = "Java";
for (int i = 0; i < text.length(); i++) {
    char c = text.charAt(i);
    System.out.println(c);  // Gibt aus: J, a, v, a
}

// Pruefung einzelner Zeichen
String eingabe = "A1B2";
if (eingabe.charAt(0) == 'A') {
    System.out.println("Beginnt mit A");
}

// Zeichen vergleichen
String code = "12345";
if (code.charAt(2) == '3') {
    System.out.println("An Position 2 steht eine 3");
}
```

Worauf achten:

- Die Nummerierung beginnt bei 0
- Die Nummer muss zwischen 0 und `text.length() - 1` liegen
- Wenn die Nummer zu groß oder zu klein ist, stürzt das Programm ab
- Du bekommst ein einzelnes Zeichen (char) zurück, keinen Text (String)

Ausgabe auf der Konsole

Mit `System.out.println()` und `System.out.print()` kannst du Text und Zahlen auf der Konsole ausgeben.

println() - Ausgabe mit Zeilenumbruch

Was macht die Methode?

Gibt Text, Zahlen oder andere Werte auf der Konsole aus und springt danach in die nächste Zeile.

Syntax:

```
System.out.println(wert);
```

Beispiele:

```
// Text ausgeben
System.out.println("Hallo Welt!");
// Ausgabe: Hallo Welt!

// Zahlen ausgeben
System.out.println(42);
// Ausgabe: 42

System.out.println(3.14);
// Ausgabe: 3.14

// Variablen ausgeben
String name = "Anna";
System.out.println(name);
// Ausgabe: Anna

int alter = 25;
System.out.println(alter);
// Ausgabe: 25

// Text und Variablen kombinieren
System.out.println("Ich bin " + alter + " Jahre alt");
// Ausgabe: Ich bin 25 Jahre alt

System.out.println("Hallo " + name + "!");
// Ausgabe: Hallo Anna!

```

```java
// Mehrere Ausgaben hintereinander
System.out.println("Erste Zeile");
System.out.println("Zweite Zeile");
System.out.println("Dritte Zeile");
// Ausgabe:
// Erste Zeile
// Zweite Zeile
// Dritte Zeile

// Berechnungen direkt ausgeben
System.out.println(5 + 3);
// Ausgabe: 8

System.out.println("Ergebnis: " + (10 * 2));
// Ausgabe: Ergebnis: 20

// Leere Zeile ausgeben
System.out.println();
// Ausgabe: (eine leere Zeile)

// Boolean-Werte ausgeben
boolean istWahr = true;
System.out.println(istWahr);
// Ausgabe: true
```

Worauf achten:

- Nach der Ausgabe springt der Cursor automatisch in die nächste Zeile
- Du kannst Text, Zahlen, Variablen und sogar Berechnungen ausgeben
- Zum Verbinden von Text und Variablen benutzt du das Plus-Zeichen (+)
- Ohne Wert macht `println()` einfach eine neue Zeile

print() - Ausgabe ohne Zeilenumbruch

Was macht die Methode?

Gibt Text, Zahlen oder andere Werte auf der Konsole aus, bleibt aber in der gleichen Zeile.

Syntax:

```
System.out.print(wert);
```

Beispiele:

```
// Text ausgeben ohne Zeilenumbruch
System.out.print("Hallo ");
System.out.print("Welt!");
// Ausgabe: Hallo Welt!

// Zahlen ausgeben
System.out.print(1);
System.out.print(2);
System.out.print(3);
// Ausgabe: 123

// Mit Leerzeichen trennen
System.out.print("Eins ");
System.out.print("Zwei ");
System.out.print("Drei");
// Ausgabe: Eins Zwei Drei

// Z hler in einer Zeile
for (int i = 1; i <= 5; i++) {
    System.out.print(i + " ");
}
// Ausgabe: 1 2 3 4 5

// Sterne in einer Reihe ausgeben
for (int i = 0; i < 10; i++) {
    System.out.print("*");
}
// Ausgabe: **********

// Kombination mit println()
System.out.print("Name: ");
System.out.println("Anna");
System.out.print("Alter: ");
System.out.println(25);
```

```
// Ausgabe:
// Name: Anna
// Alter: 25
```

Worauf achten:

- Der Cursor bleibt in der gleichen Zeile - es gibt keinen Zeilenumbruch
- Nützlich, wenn du mehrere Dinge in eine Zeile schreiben willst
- Oft wird `print()` mit `println()` kombiniert
- Denk daran, Leerzeichen einzufügen, wenn du Wörter trennen willst

Der Unterschied: print() vs. println()

Direkter Vergleich:

```
// Mit println() - jedes Mal eine neue Zeile
System.out.println("A");
System.out.println("B");
System.out.println("C");
// Ausgabe:
// A
// B
// C

// Mit print() - alles in einer Zeile
System.out.print("A");
System.out.print("B");
System.out.print("C");
// Ausgabe:
// ABC

// Gemischt - Kombination aus beiden
System.out.print("Vorname: ");
System.out.println("Max");
System.out.print("Nachname: ");
System.out.println("Mustermann");
// Ausgabe:
// Vorname: Max
// Nachname: Mustermann
```

Wann benutzt du was?

- **println()**: Wenn jede Ausgabe in einer eigenen Zeile stehen soll
- **print()**: Wenn du mehrere Dinge in einer Zeile ausgeben willst
- **Kombination**: print() für die Beschriftung, println() für den Wert

Praktisches Beispiel:

```
// Quadratzahlen ausgeben
System.out.println("Quadratzahlen von 1 bis 5:");
for (int i = 1; i <= 5; i++) {
    System.out.print(i + "   = ");
    System.out.println(i * i);
}
// Ausgabe:
// Quadratzahlen von 1 bis 5:
```

```
9  // 1   = 1
10 // 2   = 4
11 // 3   = 9
12 // 4   = 16
13 // 5   = 25
14
15 // Muster ausgeben
16 for (int zeile = 1; zeile <= 3; zeile++) {
17     for (int spalte = 1; spalte <= 5; spalte++) {
18         System.out.print("* ");
19     }
20     System.out.println();  // Neue Zeile nach jeder Reihe
21 }
22 // Ausgabe:
23 // * * * * *
24 // * * * * *
25 // * * * * *
```

ConsoleReader - Eingaben von der Konsole

Der `ConsoleReader` ist ein Werkzeug, mit dem du Eingaben vom Benutzer einlesen kannst. Um ihn zu benutzen, schreibst du am Anfang deines Programms:

```
import static prog.ConsoleReader.*;
```

Alle Eingabe-Methoden gibt es in zwei Varianten:

- Ohne Text: Wartet einfach auf die Eingabe
- Mit Text: Zeigt zuerst eine Aufforderung an (z.B. "Wie heißt du?")

readString() - String einlesen

Was macht die Methode?

Liest eine Zeile Text von der Konsole ein und gibt sie als String zurück.

Syntax:

```
String text = readString();                // Ohne Prompt
String text = readString("Eingabetext");   // Mit Prompt
```

Beispiele:

```
// Einfache Eingabe
String name = readString("Wie heisst du? ");
System.out.println("Hallo " + name);

// Mehrere Eingaben
String vorname = readString("Vorname");
String nachname = readString("Nachname");
String vollername = vorname + " " + nachname;

// Ohne Prompt
System.out.println("Gib einen Text ein:");
String eingabe = readString();

// In Schleife verwenden
String antwort;
do {
    antwort = readString("Weiter? (ja/nein)");
} while (!antwort.equals("nein"));
```

Worauf achten:

- Liest alles, was der Benutzer eingibt, bis er Enter drückt
- Du bekommst immer Text (String) zurück, auch wenn jemand "123ëingibt
- Wenn du Zahlen brauchst, benutze `readInt()` oder `readDouble()`
- Wenn der Benutzer nur Enter drückt, bekommst du einen leeren Text zurück

readInt() - Ganzzahl einlesen

Was macht die Methode?

Liest eine Ganzzahl von der Konsole ein und gibt sie als `int` zurück.

Syntax:

```
int zahl = readInt();                        // Ohne Prompt
int zahl = readInt("Gib eine Zahl ein");     // Mit Prompt
```

Beispiele:

```
// Alter einlesen
int alter = readInt("Wie alt bist du? ");
System.out.println("Du bist " + alter + " Jahre alt");

// Berechnungen
int a = readInt("Erste Zahl");
int b = readInt("Zweite Zahl");
int summe = a + b;
System.out.println("Summe: " + summe);

// Array-Groesse einlesen
int anzahl = readInt("Wie viele Zahlen? ");
int[] zahlen = new int[anzahl];
for (int i = 0; i < zahlen.length; i++) {
    zahlen[i] = readInt("Zahl " + (i + 1));
}

// Schleifensteuerung
int wiederholungen = readInt("Wie oft wiederholen? ");
for (int i = 0; i < wiederholungen; i++) {
    System.out.println("Durchlauf " + (i + 1));
}
```

Worauf achten:

- Wenn jemand Text statt einer Zahl eingibt, stürzt das Programm ab
- Funktioniert nur für ganze Zahlen von etwa -2 Milliarden bis +2 Milliarden
- Für größere Zahlen: `readLong()` benutzen
- Für Kommazahlen: `readDouble()` benutzen

readDouble() - Kommazahl einlesen

Was macht die Methode?

Liest eine Kommazahl von der Konsole ein und gibt sie als `double` zurück.

Syntax:

```
double zahl = readDouble();                    // Ohne Prompt
double zahl = readDouble("Gib eine Zahl ein"); // Mit Prompt
```

Beispiele:

```
// Preis einlesen
double preis = readDouble("Preis in Euro");
double mehrwertsteuer = preis * 0.19;
double gesamtpreis = preis + mehrwertsteuer;
System.out.println("Gesamt: " + gesamtpreis + " Euro");

// Durchschnitt berechnen
double note1 = readDouble("Note 1");
double note2 = readDouble("Note 2");
double note3 = readDouble("Note 3");
double durchschnitt = (note1 + note2 + note3) / 3.0;
System.out.println("Durchschnitt: " + durchschnitt);

// Temperatur
double celsius = readDouble("Temperatur in Celsius");
double fahrenheit = celsius * 9.0 / 5.0 + 32.0;
System.out.println(celsius + " C  = " + fahrenheit + " F ");

// Auch Ganzzahlen moeglich
double wert = readDouble("Zahl");  // Eingabe: 5
// wert = 5.0
```

Worauf achten:

- Je nach Computer funktioniert Komma `,` oder Punkt `.` als Trennzeichen
- Wenn jemand normalen Text eingibt, stürzt das Programm ab
- Du kannst auch ganze Zahlen eingeben - sie werden automatisch zu Kommazahlen
- Für die meisten Fälle ist `readDouble()` besser als `readFloat()`

readByte() - Sehr kleine Ganzzahl einlesen

Was macht die Methode?

Liest eine sehr kleine Ganzzahl von der Konsole ein und gibt sie als `byte` zurück.

Syntax:

```
byte zahl = readByte();                    // Ohne Prompt
byte zahl = readByte("Gib eine Zahl ein"); // Mit Prompt
```

Beispiele:

```
// Alter (wenn < 127)
byte alter = readByte("Alter");

// Prozent (0-100)
byte prozent = readByte("Prozent (0-100)");
```

Worauf achten:

- Wertebereich: nur -128 bis 127
- Bei zu großen Zahlen: Fehler
- Meist wird `readInt()` bevorzugt

readShort() - Kleine Ganzzahl einlesen

Was macht die Methode?

Liest eine kleine Ganzzahl von der Konsole ein und gibt sie als `short` zurück.

Syntax:

```
short zahl = readShort();                    // Ohne Prompt
short zahl = readShort("Gib eine Zahl ein"); // Mit Prompt
```

Beispiele:

```
// Jahr
short jahr = readShort("Geburtsjahr");

// Distanz in Metern
short meter = readShort("Distanz in Metern");
```

Worauf achten:

- Wertebereich: -32.768 bis 32.767
- Bei zu großen Zahlen: Fehler
- Meist wird `readInt()` bevorzugt

readLong() - Sehr große Ganzzahl einlesen

Was macht die Methode?

Liest eine sehr große Ganzzahl von der Konsole ein und gibt sie als `long` zurück.

Syntax:

```
long zahl = readLong();                    // Ohne Prompt
long zahl = readLong("Gib eine Zahl ein"); // Mit Prompt
```

Beispiele:

```
// Sehr gro e Zahlen
long weltbevoelkerung = readLong("Weltbevoelkerung");

// Millisekunden
long millis = readLong("Millisekunden");
```

Worauf achten:

- Für sehr große Zahlen (größer als ca. 2 Milliarden)
- Wertebereich: ca. -9,2 $\times$ 10^{18} bis 9,2 $\times$ 10^{18}
- Für normale Zahlen: `readInt()` verwenden

readFloat() - Einfache Kommazahl einlesen

Was macht die Methode?

Liest eine Kommazahl mit einfacher Genauigkeit von der Konsole ein und gibt sie als `float` zurück.

Syntax:

```
float zahl = readFloat();                    // Ohne Prompt
float zahl = readFloat("Gib eine Zahl ein"); // Mit Prompt
```

Beispiele:

```
// Gewicht
float gewicht = readFloat("Gewicht in kg");

// Groesse
float groesse = readFloat("Groesse in m");
float bmi = gewicht / (groesse * groesse);
```

Worauf achten:

- Weniger genau als `double` (ca. 6-7 Dezimalstellen)
- Meist wird `readDouble()` bevorzugt
- Dezimaltrennzeichen: Komma oder Punkt (je nach System)

readIntArray() - Ganzzahl-Array einlesen

Was macht die Methode?

Liest mehrere Ganzzahlen auf einmal ein und gibt sie als Array zurück. Die Zahlen müssen mit Leerzeichen getrennt werden.

Syntax:

```
int[] zahlen = readIntArray();                    // Ohne Prompt
int[] zahlen = readIntArray("Gib Zahlen ein");    // Mit Prompt
```

Beispiele:

```
// Mehrere Zahlen auf einmal einlesen
int[] noten = readIntArray("Noten (durch Leerzeichen getrennt)");
// Eingabe: 1 2 3 2 1
// noten = {1, 2, 3, 2, 1}

// Durchschnitt berechnen
int summe = 0;
for (int note : noten) {
    summe += note;
}
double durchschnitt = (double) summe / noten.length;
System.out.println("Durchschnitt: " + durchschnitt);

// Array direkt ausgeben
int[] zahlen = readIntArray("Zahlen");
System.out.println("Anzahl: " + zahlen.length);
for (int i = 0; i < zahlen.length; i++) {
    System.out.println("Zahl " + (i + 1) + ": " + zahlen[i]);
}
```

Worauf achten:

- Die Zahlen müssen mit Leerzeichen getrennt sein (z.B. "1 2 3“)
- Die Größe des Arrays passt sich automatisch an die Anzahl der Zahlen an
- Wenn jemand Text eingibt, stürzt das Programm ab
- Alle Zahlen müssen in einer Zeile stehen (Enter beendet die Eingabe)

readDoubleArray() - Kommazahl-Array einlesen

Was macht die Methode?

Liest mehrere Kommazahlen von der Konsole ein und gibt sie als `double[]`-Array zurück. Die Zahlen müssen durch Leerzeichen getrennt eingegeben werden.

Syntax:

```
double[] zahlen = readDoubleArray();                    // Ohne Prompt
double[] zahlen = readDoubleArray("Gib Zahlen ein");    // Mit Prompt
```

Beispiele:

```
// Preise einlesen
double[] preise = readDoubleArray("Preise");
// Eingabe: 9.99 14.50 7.25
// preise = {9.99, 14.50, 7.25}

// Gesamtpreis berechnen
double gesamt = 0.0;
for (double preis : preise) {
    gesamt += preis;
}
System.out.println("Gesamt: " + gesamt + " Euro");

// Temperaturwerte
double[] temps = readDoubleArray("Temperaturen");
double min = temps[0];
double max = temps[0];
for (double temp : temps) {
    if (temp < min) min = temp;
    if (temp > max) max = temp;
}
System.out.println("Min: " + min + ", Max: " + max);
```

Worauf achten:

- Zahlen müssen durch **Leerzeichen** getrennt werden
- Dezimaltrennzeichen: Komma oder Punkt (je nach System)
- Array-Größe ergibt sich automatisch aus der Anzahl der Eingaben
- Bei ungültiger Eingabe: Programm bricht ab

readStringArray() - String-Array einlesen

Was macht die Methode?

Liest mehrere Wörter von der Konsole ein und gibt sie als `String[]`-Array zurück. Die Wörter müssen durch Leerzeichen getrennt eingegeben werden.

Syntax:

```
String[] woerter = readStringArray();                    // Ohne Prompt
String[] woerter = readStringArray("Gib Woerter ein");   // Mit Prompt
```

Beispiele:

```
// Namen einlesen
String[] namen = readStringArray("Namen");
// Eingabe: Anna Bob Clara
// namen = {"Anna", "Bob", "Clara"}

// Alle Namen ausgeben
System.out.println("Teilnehmer:");
for (String name : namen) {
    System.out.println("- " + name);
}

// Woerter zaehlen
String[] woerter = readStringArray("Satz");
System.out.println("Anzahl Woerter: " + woerter.length);

// Laengstes Wort finden
String[] text = readStringArray("Text");
String laengstes = text[0];
for (String wort : text) {
    if (wort.length() > laengstes.length()) {
        laengstes = wort;
    }
}
System.out.println("Laengstes Wort: " + laengstes);
```

Worauf achten:

- Die Wörter müssen mit Leerzeichen getrennt sein
- Wenn du ganze Sätze einlesen willst, benutze lieber `readString()`
- Die Größe des Arrays passt sich automatisch an die Anzahl der Wörter an
- Für Sätze: `readString()` benutzen und dann mit `.split()` aufteilen

Weitere Array-Methoden

Der ConsoleReader bietet auch Methoden für andere Datentypen:

- `readByteArray()` - Liest `byte[]`-Array ein
- `readShortArray()` - Liest `short[]`-Array ein
- `readLongArray()` - Liest `long[]`-Array ein
- `readFloatArray()` - Liest `float[]`-Array ein

Diese funktionieren analog zu `readIntArray()` und `readDoubleArray()`.

Beispiel:

```
// Byte-Array
byte[] bytes = readByteArray("Bytes");

// Long-Array fuer grosse Zahlen
long[] ids = readLongArray("IDs");

// Float-Array
float[] gewichte = readFloatArray("Gewichte");
```

Bildverarbeitung

Die `Picture`-Klasse ist ein Werkzeug zum Arbeiten mit Bildern. Bilder werden als zweidimensionale Arrays gespeichert, wobei jeder Wert eine Farbe darstellt. Um sie zu benutzen, schreibst du am Anfang:

```
1 import static prog.Picture.*;
```

show() - Bild anzeigen

Was macht die Methode?

Zeigt ein Bild (als `int[][]`-Array) in einem Fenster an.

Syntax:

```
show(bildArray);                    // Ohne Fenstertitel
show(bildArray, "Fenstertitel");    // Mit Fenstertitel
```

Beispiele:

```
// Bild aus Datei laden und anzeigen
int[][] bild = loadFromFileSystem("pfad/zum/bild.jpg");
show(bild);

// Mit eigenem Fenstertitel
int[][] foto = loadFromFileSystem("foto.png");
show(foto, "Mein Urlaubsfoto");

// Mehrere Bilder anzeigen
int[][] bild1 = loadResource("bild1.jpg");
int[][] bild2 = loadResource("bild2.jpg");
show(bild1, "Original");
show(bild2, "Bearbeitet");

// Selbst erstelltes Bild anzeigen
int[][] neuBild = new int[100][100];
for (int y = 0; y < neuBild.length; y++) {
    for (int x = 0; x < neuBild[0].length; x++) {
        neuBild[y][x] = color(255, 0, 0);  // Rot
    }
}
show(neuBild, "Rotes Quadrat");
```

Worauf achten:

- Das Array hat das Format `int[Höhe][Breite]` - erst Höhe, dann Breite
- Jede Zahl im Array stellt eine Farbe dar
- Das Fenster bleibt offen, bis du es schließt
- Wenn du keinen Titel angibst, wird ein Standardtitel verwendet

loadFromFileSystem() - Bild von Festplatte laden

Was macht die Methode?

Lädt ein Bild von der Festplatte und gibt es als `int[][]`-Array zurück.

Syntax:

```
int[][] bild = loadFromFileSystem("pfad/zur/datei.jpg");
```

Beispiele:

```
// Absoluter Pfad
int[][] bild1 = loadFromFileSystem("/home/user/bilder/foto.jpg");
show(bild1);

// Relativer Pfad (relativ zum Projektverzeichnis)
int[][] bild2 = loadFromFileSystem("bilder/landschaft.png");
show(bild2, "Landschaft");

// Bild laden und Informationen ausgeben
int[][] foto = loadFromFileSystem("foto.jpg");
int hoehe = foto.length;
int breite = foto[0].length;
System.out.println("Bildgroesse: " + breite + " x " + hoehe);

// Bild laden und verarbeiten
int[][] original = loadFromFileSystem("bild.jpg");
int[][] bearbeitet = new int[original.length][original[0].length];
// ... Bildverarbeitung ...
show(original, "Original");
show(bearbeitet, "Bearbeitet");
```

Worauf achten:

- Funktioniert mit den gängigen Bildformaten: JPG, PNG, BMP, GIF
- Du kannst den vollständigen Pfad oder einen relativen Pfad angeben
- Wenn die Datei nicht existiert, stürzt das Programm ab
- Für Bilder, die zu deinem Programm gehören: benutze `loadResource()`

loadResource() - Bild aus Projekt laden

Was macht die Methode?

Lädt ein Bild, das zu deinem Projekt gehört, und gibt es als Array zurück. Das Bild muss im selben Ordner liegen wie dein Programm.

Syntax:

```
int[][] bild = loadResource("dateiname.jpg");
```

Beispiele:

```
// Bild direkt aus dem Package laden
int[][] logo = loadResource("logo.png");
show(logo, "Logo");

// Mehrere Bilder laden
int[][] bild1 = loadResource("bild1.jpg");
int[][] bild2 = loadResource("bild2.jpg");
show(bild1, "Bild 1");
show(bild2, "Bild 2");

// Bild laden und Groesse pruefen
int[][] foto = loadResource("foto.png");
if (foto.length > 0 && foto[0].length > 0) {
    System.out.println("Bild erfolgreich geladen");
    show(foto);
}

// Typische Verwendung in einfachen Projekten
// Dateistruktur:
// src/
//   prog/
//     Main.java
//     bild.jpg  <- Bild liegt direkt neben der Klasse

// Variante 1: Relativ
int[][] bild = loadResource("bild.jpg");
show(bild);

// Variante 2: Absolut vom Classpath-Root
int[][] bild2 = loadResource("/prog/bild.jpg");
show(bild2);
```

Worauf achten:

- Nur für Bilder benutzen, die zu deinem Projekt gehören
- Das Bild muss im selben Ordner liegen wie dein Programm
- Gib nur den Dateinamen an, keinen Pfad
- Für Bilder von woanders auf deinem Computer: benutze `loadFromFileSystem()`

color() - Farbwert erstellen

Was macht die Methode?

Erstellt einen RGB-Farbwert aus Rot-, Grün- und Blau-Anteilen.

Syntax:

```
int farbwert = color(rot, gruen, blau);  // Werte: 0-255
```

Beispiele:

```
// Grundfarben
int rot = color(255, 0, 0);
int gruen = color(0, 255, 0);
int blau = color(0, 0, 255);

// Mischfarben
int gelb = color(255, 255, 0);      // Rot + Gruen
int magenta = color(255, 0, 255);   // Rot + Blau
int cyan = color(0, 255, 255);      // Gruen + Blau

// Graustufen
int schwarz = color(0, 0, 0);
int weiss = color(255, 255, 255);
int grau = color(128, 128, 128);

// Bild mit Farbe fuellen
int[][] bild = new int[100][200];
int orange = color(255, 165, 0);
for (int y = 0; y < bild.length; y++) {
    for (int x = 0; x < bild[0].length; x++) {
        bild[y][x] = orange;
    }
}
show(bild, "Orange");

// Farbverlauf erstellen
int[][] verlauf = new int[256][100];
for (int y = 0; y < verlauf.length; y++) {
    int farbwert = color(y, 0, 255 - y);  // Von Blau zu Rot
    for (int x = 0; x < verlauf[0].length; x++) {
        verlauf[y][x] = farbwert;
    }
}
show(verlauf, "Farbverlauf");
```

```
35
36 // Pixel-Farbe setzen
37 int[][] foto = loadResource("foto.jpg");
38 foto[50][100] = color(255, 0, 0);  // Roten Punkt setzen
39 show(foto);
```

Worauf achten:

- Jede Farbe (Rot, Grün, Blau) kann einen Wert von 0 bis 255 haben
- 0 bedeutet "gar nichts von dieser Farbe", 255 bedeutet mmaximale Helligkeit"
- RGB steht für: Rot, Grün, Blau
- Die Methode gibt eine einzige Zahl zurück, die alle drei Farben enthält

red(), green(), blue() - Farbkanäle extrahieren

Was machen die Methoden?

Holen den Rot-, Grün- oder Blau-Anteil aus einer Farbe heraus.

Syntax:

```
int rotWert = red(farbwert);      // Rot-Anteil (0-255)
int gruenWert = green(farbwert);  // Gruen-Anteil (0-255)
int blauWert = blue(farbwert);    // Blau-Anteil (0-255)
```

Beispiele:

```
// Farbkan le eines Farbwerts extrahieren
int orange = color(255, 165, 0);
int r = red(orange);      // r = 255
int g = green(orange);    // g = 165
int b = blue(orange);     // b = 0
System.out.println("RGB: " + r + ", " + g + ", " + b);

// Pixel-Farbe analysieren
int[][] bild = loadResource("foto.jpg");
int pixel = bild[50][100];  // Pixel an Position (100, 50)
System.out.println("Rot: " + red(pixel));
System.out.println("Gruen: " + green(pixel));
System.out.println("Blau: " + blue(pixel));

// Bild in Graustufen umwandeln
int[][] original = loadResource("bild.jpg");
int[][] graustufen = new int[original.length][original[0].length];
for (int y = 0; y < original.length; y++) {
    for (int x = 0; x < original[0].length; x++) {
        int pixel = original[y][x];
        int r = red(pixel);
        int g = green(pixel);
        int b = blue(pixel);
        int grau = (r + g + b) / 3;  // Durchschnitt
        graustufen[y][x] = color(grau, grau, grau);
    }
}
show(original, "Original");
show(graustufen, "Graustufen");

// Nur Rot-Kanal behalten
int[][] nurRot = new int[original.length][original[0].length];
```

```
for (int y = 0; y < original.length; y++) {
    for (int x = 0; x < original[0].length; x++) {
        int r = red(original[y][x]);
        nurRot[y][x] = color(r, 0, 0);
    }
}
show(nurRot, "Nur Rot-Kanal");

// Farbkan le vertauschen (Rot <-> Blau)
int[][] vertauscht = new int[original.length][original[0].length];
for (int y = 0; y < original.length; y++) {
    for (int x = 0; x < original[0].length; x++) {
        int pixel = original[y][x];
        int r = red(pixel);
        int g = green(pixel);
        int b = blue(pixel);
        vertauscht[y][x] = color(b, g, r);  // Blau und Rot tauschen
    }
}
show(vertauscht, "Vertauschte Kan le");

// Helligkeit eines Pixels berechnen
int pixel = bild[0][0];
int helligkeit = (red(pixel) + green(pixel) + blue(pixel)) / 3;
System.out.println("Helligkeit: " + helligkeit);
```

Worauf achten:

- Du bekommst immer eine Zahl zwischen 0 und 255 zurück
- Sehr nützlich, wenn du Bilder bearbeiten willst
- Zusammen mit `color()` kannst du einzelne Farbanteile ändern
- Häufige Anwendung: Bilder in Graustufen umwandeln oder Farbfilter erstellen

Häufige Fehler bei Strings

String.length ohne Klammern

```
// FALSCH
String text = "Hallo";
int laenge = text.length;  // Fehler: Strings brauchen ()

// RICHTIG
int laenge = text.length();    // String MIT Klammern
```

Index-Grenzen bei charAt

```
String wort = "Test";  // length = 4, Indizes: 0, 1, 2, 3
char c = wort.charAt(4);  // Fehler! Index 4 existiert nicht

// RICHTIG
char letztes = wort.charAt(wort.length() - 1);  // Index 3
char erstes = wort.charAt(0);                   // Index 0
```

char vs. String

```
String text = "A";
char buchstabe = 'A';

// FALSCH
if (text.charAt(0) == "A") { }  // Vergleicht char mit String

// RICHTIG
if (text.charAt(0) == 'A') { }  // char mit char vergleichen
```